

Browser AI Agent Platform

The Complete Project Guide — Every Piece Explained, End to End

1. The 30-Second Version

This is a mini version of tools like OpenAI Operator or Manus. You type a task in plain English — "find the cheapest iPhone," "summarize this page" — and the system uses an AI model to decide what actions to take, then a browser automation tool (Playwright) to actually perform those actions in a real, live browser. It reads pages, clicks buttons, fills search boxes, and repeats this in a loop until the task is done, then returns a full report with its reasoning and screenshots for every step.

It's a complete, working full-stack application: a React frontend, a FastAPI backend, an AI agent engine at the core, and a deployment pipeline (Docker, Render, Vercel) — not just a script.

2. The Three Things Happening Every Time a Task Runs

- **Understand** — The AI model reads the plain-English task and the current page, and decides what the single next action should be.
- **Act** — Playwright, a browser automation library, actually performs that action in a real browser: clicking, typing, navigating, or reading text.
- **Report** — Once the AI decides the task is complete, everything — every step, every screenshot, the final answer — gets saved and shown back to the user.

3. Full Architecture — How Every Piece Connects

Here is the complete chain of responsibility, from the moment a person types a task to the moment they see a result:

Layer	What it does
1. React frontend (App.jsx)	Renders the form (Task + Starting URL inputs), sends the request, shows loading/error/result states, and displays the History tab.
2. FastAPI backend (main.py)	Receives the HTTP request, validates it, calls the agent engine, and returns the result as JSON. Also serves the /reports endpoints for browsing past runs.
3. Agent engine (agent/core.py)	The actual brain-and-hands loop: reads the page, asks the AI model for a decision, executes that decision with Playwright, and repeats until done.

Layer	What it does
4. Groq API (external)	Hosts the LLM (openai/gpt-oss-120b) that reasons about the task and decides each action.
5. Playwright (library)	Controls a real, actual browser (Chromium) — navigating, clicking, typing, screenshotting.
6. reports/ folder (disk)	Where every run's report.json, report.md, and per-step screenshots get saved.

The key architectural idea: each layer only talks to the one directly next to it. The frontend never talks to Playwright or Groq directly — it only ever talks to the FastAPI backend. This means any piece can be swapped later (a different frontend, a different LLM provider) without rewriting everything else.

4. Full Tech Stack — What Each Tool Is For

Tool	Category	What it's for
React (Vite)	Frontend	Builds the actual webpage — form, loading states, results, history browser.
FastAPI	Backend framework	Python web framework exposing the agent as HTTP endpoints (/run-task, /reports).
Playwright	Browser automation	Controls a real, automated Chromium browser — navigate, click, fill, screenshot.
Groq API	AI inference provider	Hosts and serves the LLM used for decision-making, on a free tier.
openai/gpt-oss-120b	The LLM itself	The model reading page state and deciding the next action, in structured JSON.
Pydantic	Data validation	Defines and auto-validates the shape of incoming API requests (TaskRequest).
python-dotenv	Secrets management	Loads the GROQ_API_KEY from a local .env file, kept out of source control.
Docker	Packaging/deployment	Bundles the backend (with Playwright's browser binaries) into a portable container.
Render	Backend hosting	Runs the Dockerized FastAPI backend publicly, on a free tier.
Vercel	Frontend hosting	Hosts the built React app publicly, on a free tier.
Git / GitHub	Version control	Tracks every change, hosts the public source repository.

5. Project Folder Structure

```
browser-ai-agent/  
main.py <- FastAPI backend: API endpoints  
agent/  
__init__.py <- marks 'agent' as an importable package  
core.py <- the actual agent loop: the engine  
frontend/  
src/  
App.jsx <- the React UI: form, results, history  
App.css <- all visual styling  
reports/ <- generated per task run (not committed to git)  
practice-days/ <- early learning scripts, kept for reference  
.env <- GROQ_API_KEY (never committed)  
.gitignore  
requirements.txt <- Python dependencies  
Dockerfile <- backend container definition  
README.md
```

6. The Agent Engine — agent/core.py (The Heart of the Project)

This file contains three functions. Together they implement the full **observe** → **plan** → **act** loop — the core idea the entire platform is built around.

- **get_page_summary(page)** — OBSERVE: turns a real webpage into text the AI can reason about
- **ask_ai(task, page_summary, history)** — PLAN: asks the LLM to decide the next single action
- **run_agent(task, start_url, ...)** — ACT + LOOP: executes the decision with Playwright, repeats

6a. get_page_summary() — Turning a Webpage Into AI-Readable Text

An AI model can't "see" a webpage the way a human does, and raw HTML is too messy and huge to send directly. This function's job is to compress a real, live page down into a short, structured text description the AI can actually reason about.

```
def get_page_summary(page, max_elements=25):
    page.evaluate("""
    () => {
      const els = document.querySelectorAll('input, button, a, select, textarea');
      let counter = 0;
      els.forEach((el) => {
        const rect = el.getBoundingClientRect();
        const isVisible = rect.width > 0 && rect.height > 0 && ...
        if (isVisible) {
          el.setAttribute('data-agent-id', counter);
          counter++;
        }
      });
    }
    """)
    visible_text = page.inner_text("body")[:2500]
    # ... score and rank elements, build the final text summary ...
```

page.evaluate

Runs raw JavaScript directly inside the live browser page. This is the only way to reach into the page's actual DOM and tag elements — Playwright's Python API alone can't inject custom attributes onto elements.

querySelectorAll

Finds every input, button, link, dropdown, and text area currently in the page's HTML — these are the only kinds of elements a user could actually interact with.

Visibility check

Filters out elements that exist in the HTML but aren't actually visible on screen (zero size, hidden, or display:none). This exists because an earlier version tried to click an invisible element and Playwright correctly refused, causing a 30-second timeout.

data-agent-id

A custom attribute this project invents and injects itself — not a real web standard. Every visible interactive element gets stamped with a simple number. This lets the AI refer to "element 4" instead of writing real CSS, which turned out to be unreliable (see Section 9).

`visible_text (page
text)`

Grabs the page's visible text content, capped at 2500 characters so the AI's prompt doesn't become enormous and expensive on large pages.

`Scoring elements`

After tagging, every element is given a numeric score — elements containing price-like text or sort/filter keywords ("low to high", "sort") score higher than plain navigation links. Only the top `max_elements` survive into the final summary, so the AI sees the most relevant controls on cluttered pages instead of random nav links.

6b. ask_ai() — The Decision-Making Call

This function sends the task, the current page summary, and the history of actions already taken to the LLM, and gets back a single decision in structured JSON.

```
def ask_ai(task, page_summary, history):
    history_text = "\n".join(f"{i+1}. {h}" for i, h in enumerate(history))
    prompt = f"""You are a browser automation agent ...
    Task: {task}
    Actions already taken so far: {history_text}
    Current page: {page_summary}

    You must choose exactly ONE of these actions:
    - navigate / fill / press / click / click_by_text / done

    Respond ONLY with valid JSON: {"action":..., "id":..., "value":..., "answer":...,
    "reasoning":...}
    """

    response = client.chat.completions.create(
        model="openai/gpt-oss-120b",
        messages=[{"role": "user", "content": prompt}]
    )
    return json.loads(response.choices[0].message.content)
```

history_text

A running list of every action already taken, formatted as numbered plain text and inserted into the prompt. Without this, the AI has no memory between steps and will repeat the same action forever (this was a real bug — see Section 9).

The action menu

The prompt explicitly lists every action the AI is allowed to choose: navigate, fill, press, click, click_by_text, or done. Giving an explicit, complete menu — instead of one example — was itself a fix for a real bug where the AI defaulted to one action even when it wasn't the right choice (Section 9).

Using id, not CSS

The AI is told to reference elements using the plain number shown next to them (from data-agent-id), never to write its own CSS. An earlier version let the AI write selectors freely, and it invented fake, nonexistent selector syntax.

Task-specific rules in the prompt

The full prompt (not fully shown above) also contains explicit rules for specific task types — e.g. never conclude "cheapest" without sorting results first. These rules were added one at a time, each fixing a specific observed failure (Section 9).

The Groq API call

The actual network call to Groq's API, specifying which model to use and sending the full prompt as a single user message.

json.loads(...)

Converts the AI's raw text reply into a real Python dictionary so the rest of the code can access decision["action"], decision["id"], etc. directly.

6c. run_agent() — The Loop That Ties It All Together

This is the actual observe-plan-act loop. It opens a browser, repeatedly calls `get_page_summary()` and `ask_ai()`, executes whatever action comes back using Playwright, and stops when the AI says the task is done or a step limit is reached.

```
def run_agent(task, start_url, max_steps=10, headless=False):
    run_id = datetime.now().strftime("%Y%m%d_%H%M%S")
    run_folder = f"reports/{run_id}"
    ...
    with sync_playwright() as p:
        browser = p.chromium.launch(headless=headless)
        page = browser.new_page()
        page.goto(start_url, timeout=30000, wait_until="domcontentloaded")
        history = []

        for step in range(max_steps):
            summary = get_page_summary(page)
            decision = ask_ai(task, summary, history)

            if decision["action"] == "done":
                report["final_answer"] = decision["answer"]
                break

            try:
                # execute navigate / fill / press / click / click_by_text
                # append what happened to history
            except Exception as e:
                # log the failure into history instead of crashing

        browser.close()

        # save report.json and report.md to disk
        return report
```

run_id / run_folder

Every run gets a unique, timestamped folder, so multiple task runs never overwrite each other and every past run stays browsable in the History tab.

Browser lifecycle

Opens a real Chromium browser for the duration of this one task, and guarantees it's properly closed afterward, even if something fails partway through.

headless parameter

When True (used by the deployed API), the browser runs invisibly — no window pops up, faster and lighter, appropriate for a server with no one watching. When False (used for local learning/debugging), a real window opens so you can watch the agent work.

The for loop + max_steps

The actual repeating cycle: observe (`get_page_summary`), plan (`ask_ai`), and — unless the answer is 'done' — act, then loop again. `max_steps` is a hard safety limit so a confused agent can't run forever.

Checking for done

The loop's only real exit condition besides running out of steps. Everything before this in the prompt is designed to help the AI recognize this moment correctly and not too early or too late.

`try/except block`

If Playwright can't perform the chosen action (element not found, not clickable, etc.), the failure is caught, logged into history so the AI knows not to retry the exact same thing, and the loop continues instead of crashing the whole task.

`report.json` and
`report.md`

The same data saved twice: JSON for code to read later (e.g. the `/reports` API endpoints), Markdown for a human to open and read directly.

7. The Backend — main.py

This file turns the agent engine into a real web service. It doesn't contain any agent logic itself — it only receives requests, calls `agent/core.py`, and returns responses.

```
app = FastAPI(title="Browser AI Agent API")
app.add_middleware(CORSMiddleware, allow_origins=["*"], ...)

class TaskRequest(BaseModel):
    task: str
    start_url: str

@app.post("/run-task")
def run_task(request: TaskRequest):
    report = run_agent(task=request.task, start_url=request.start_url, headless=True)
    return report

@app.get("/reports")
def list_reports():
    # returns lightweight summaries of every past run

@app.get("/reports/{run_id}")
def get_report(run_id: str):
    # returns one full report by its id
```

`CORSMiddleware`

Browsers block a webpage on one address (e.g. the React app) from calling an API on a different address (the FastAPI backend) by default, as a security measure. This middleware explicitly allows that cross-origin communication.

`TaskRequest` model

Defines the exact shape of a valid incoming request: `task` and `start_url`, both required text fields. FastAPI uses this to automatically reject malformed requests before the route function even runs.

`@app.post("/run-task")`

A decorator registering this function to handle POST requests at `/run-task`. `request: TaskRequest` means FastAPI validates and parses the incoming JSON body into a ready-to-use object automatically.

`headless=True`

The deployed API always runs the browser invisibly — there's no one at a server to watch a browser window, and headless mode is faster and lighter.

`Synchronous behavior`

The HTTP request doesn't return until the entire multi-step agent loop finishes — meaning the caller simply waits, sometimes 15-40+ seconds. This is a known, acceptable limitation for the current MVP; the natural next step would be a background task queue (Celery + Redis) so the API can respond immediately with a task ID while the agent runs separately.

`/reports` and `/reports/{run_id}`

Two read-only endpoints powering the frontend's History tab: one lists all past runs with a lightweight summary, the other fetches one run's complete detail by its folder/run ID.

8. The Frontend — App.jsx and App.css

The frontend is a single-page React app with two views, toggled by a simple tab switcher: "New Task" and "History".

Key React concepts used in App.jsx

Concept	What it does here
useState	React's way of storing values that can change and trigger a re-render when they do — used here for the task text, URL text, loading flag, result data, error message, and which view/report is currently selected.
useEffect	Runs a piece of code automatically whenever something specific changes — used here so that switching to the History tab automatically triggers loadHistory() without needing a separate manual click handler.
fetch + async/await	The actual network call to the backend. fetch(...) sends the HTTP request; await pauses execution until the response arrives; response.json() parses the JSON body into a usable JavaScript object.
Conditional rendering	React's pattern for showing/hiding UI based on state — the loading indicator, error message, and result card only render when their corresponding state variable is truthy.
className	How React elements reference CSS classes defined in App.css — equivalent to the class attribute in plain HTML.

The core network call

```
const runTask = async () => {
  setLoading(true)
  try {
    const response = await fetch(`${API_URL}/run-task`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ task, start_url: startUrl }),
    })
    const data = await response.json()
    setResult(data)
  } catch (err) {
    setError('Could not reach the agent server. Is it running?')
  } finally {
    setLoading(false)
  }
}
```

This function is triggered by the Run Task button. It sets a loading state, sends the task and URL to the backend's /run-task endpoint as JSON, waits for the full report to come back, and stores it in state so it renders on screen. The try/catch/finally ensures the loading spinner always turns off and a clear error shows if the backend is unreachable — exactly the behavior confirmed when the backend was deliberately

stopped during testing.

App.css — Design Approach

The final design intentionally uses a small, restrained palette: a light neutral background, white cards, dark readable text, and a single blue accent color (#2563eb) used consistently for buttons, active tab states, focus rings, and links. Status badges (success/incomplete/max_steps_reached) each get a distinct soft color so a run's outcome is readable at a glance. Earlier iterations experimented with a dark glassmorphism theme (blurred gradient panels, floating background blobs) before settling on this simpler, higher-contrast version for readability and reliability.

9. The Real Bugs Found and Fixed (The Most Interview-Worthy Part)

This project wasn't built by writing correct code the first time — it was built by hitting real failure modes and fixing them one at a time. This is genuinely the most valuable part to be able to explain clearly, since it demonstrates actual engineering judgment rather than just following steps.

Infinite fill loop

What happened: The agent repeated "fill the search box" forever.

Root cause: It had no memory of past actions and couldn't see the input's current value, so every step looked identical to it.

Fix: Added a history list passed into every prompt, and showed each input's actual current value.

Invented fake CSS selector

What happened: A click failed with a 30-second timeout on a nonsense selector.

Root cause: The AI was asked to write CSS freely, and it imitated the shape of the example format it was shown instead of producing valid syntax.

Fix: Replaced free-form selectors with simple numbered IDs (data-agent-id) — the AI just picks a number.

Clicking an invisible element

What happened: A click silently retried for 30+ seconds and then failed.

Root cause: The tagged element existed in the HTML but had zero size / was hidden — a real user could never have clicked it either.

Fix: Added a visibility check (getBoundingClientRect + computed style) before tagging any element.

Confidently wrong price answer

What happened: The agent said the first product it saw was the "cheapest," and it wasn't.

Root cause: Search results were sorted by relevance, not price, but nothing told the AI to check that before concluding.

Fix: Prioritized sort-control elements in the page summary, and added an explicit prompt rule requiring sorting or comparison before concluding on price tasks.

Couldn't click the sort control

What happened: Even after the fix above, the AI couldn't reach the "Price -- Low to High" control.

Root cause: That control likely wasn't a standard input/button/a tag, so it never got a numbered id at all.

Fix: Added a click_by_text fallback action, letting the AI click by visible text regardless of HTML tag.

Model deprecation (llama-3.3-70b-versatile)

What happened: The API started returning a 404 model_not_found error.

Root cause: Groq officially decommissioned that model in August 2026; it wasn't a bug in the code at all.

Fix: Swapped to openai/gpt-oss-120b, confirmed still free-tier eligible, and re-verified prompt behavior.

The single biggest lesson across all of these: a fluent, well-formatted, confidently-reasoned AI response is not the same thing as a correct one. Every fix above came from actually checking the agent's output against reality (a screenshot, an error log) rather than trusting that valid JSON meant a valid answer.

10. Deployment — Docker, Render, Vercel

Docker

The backend is packaged using Microsoft's official Playwright Python base image (rather than a plain Python image), because Playwright needs real browser binaries and OS-level libraries that are painful to install manually. The Dockerfile copies the project in, installs Python dependencies from requirements.txt, and runs uvicorn on container start. This means the backend runs identically on any machine or cloud provider, without anyone needing to manually install Python, Playwright, or system dependencies by hand.

Render (backend hosting)

Render builds and runs the Dockerfile directly, on a free tier. The GROQ_API_KEY is set as an environment variable in Render's dashboard — never baked into the Docker image itself, keeping the secret out of both git and the built container.

Vercel (frontend hosting)

Vercel builds and hosts the React app directly from the frontend/ folder, auto-detecting the Vite framework. The frontend's API_URL constant points to the live Render backend URL rather than localhost once deployed.

CORS in production

Locally, CORS allows all origins (allow_origins=["*"]) for simplicity. Once a real frontend URL exists, this should be tightened to only that specific domain, so no other website could call the API on a visitor's behalf.

11. Known Limitations (Good to State Proactively)

- **Single-user, no authentication** — anyone with the URL can run tasks and see all history.
- **Synchronous task execution** — the API blocks until the full agent loop finishes; no background queue (Celery/Redis) yet, so it can't easily handle many concurrent tasks.
- **No automated correctness verification** — answers are only as reliable as the prompt rules written for specific failure patterns already discovered; a genuinely novel task type could still fail in an undetected way.
- **Vulnerable to bot detection** — sites like Google actively block automated browsers; the agent works best on sites that don't aggressively fight automation.
- **File-based storage** — reports are saved as JSON files on disk, not in a real database, which won't survive a redeploy on most hosting platforms and doesn't support real querying/filtering.

12. Interview / Explaining-to-a-Friend Cheat Sheet

If you only remember five things, remember these:

- **What it is:** a browser agent — an LLM decides actions, Playwright executes them in a real browser, in a repeating observe-plan-act loop, until the task is done.
- **How the AI is grounded to the page:** every visible interactive element gets tagged with a numbered ID via injected JavaScript; the AI picks a number instead of writing CSS, which is far more reliable.
- **The single best story to tell:** the agent once confidently answered "cheapest iPhone" using an unsorted, relevance-ranked results page — a fluent, valid, wrong answer. Fixed by prioritizing sort controls in the page summary and adding an explicit rule requiring sorting/comparison first.
- **The architecture in one line:** React talks only to FastAPI; FastAPI calls a standalone, framework-agnostic agent/core.py module; that module is the only thing touching Playwright and Groq.
- **What's honestly still missing:** auth, background/async execution for concurrent users, and any automated correctness-checking beyond the specific rules already discovered through testing.

If asked "walk me through the architecture"

"The React frontend sends a task and URL to a FastAPI backend. FastAPI calls a single function, `run_agent`, which lives in its own module separate from the web framework. That function opens a real browser with Playwright, and loops: read the page, ask an LLM what to do next, do it, repeat, until the model says the task is done. Everything gets saved as a report with screenshots, which the frontend can also browse later through a history view."

If asked "why is the agent logic in its own module instead of inside the API file?"

"Because it needed to be callable from more than one place — a standalone test script during development, and later the FastAPI route, without duplicating logic or awkwardly importing from the web layer. Keeping `run_agent` as a plain, framework-agnostic function means the web server is just one caller among possibly several."

13. Full Glossary

Term	Meaning
Agent	A program that takes actions toward a goal (like clicking, navigating) rather than just responding with text.
Observe → Plan → Act loop	The repeating cycle at the core of this project: read the current state, decide one action, execute it, repeat.
LLM (Large Language Model)	The AI model (openai/gpt-oss-120b, via Groq) that reads context and decides the next action.
Grounding	Connecting an AI's abstract decision to a real, concrete element on the actual page — done here via numbered element IDs.

Term	Meaning
Prompt	The full block of instructions and context sent to the LLM on every decision-making call.
Structured output / JSON mode	Requiring the AI to respond in a strict, parseable format instead of free-form text.
Action space	The complete list of actions the agent is allowed to choose from at any step.
Playwright	The library controlling a real, automated browser: navigate, click, fill, screenshot.
data-agent-id	A custom HTML attribute this project injects to let the AI reference elements by number.
Headless browser	A browser running with no visible window — used in production for speed and to run on a server with no display.
FastAPI	The Python web framework exposing the agent as HTTP endpoints.
Endpoint	A specific URL + HTTP method combination a server knows how to respond to (e.g. POST /run-task).
CORS	A browser security policy blocking cross-origin requests unless explicitly allowed — handled via CORSMiddleware.
Pydantic / BaseModel	The library FastAPI uses to define and auto-validate expected request data shapes.
Synchronous endpoint	An endpoint where the caller waits until all the work is fully done before getting a response.
React state (useState)	React's mechanism for values that change over time and trigger UI re-renders.
useEffect	A React hook that runs code automatically in response to a value changing.
Docker / Dockerfile	A way of packaging an application with everything it needs so it runs identically anywhere.
Render / Vercel	Free-tier hosting platforms used for the backend (Render, via Docker) and frontend (Vercel, via Vite build) respectively.

This document is meant as a complete reference — re-read Sections 6 and 9 closely before an interview, since decision-making logic and the real bugs found are what tend to generate the most follow-up questions.